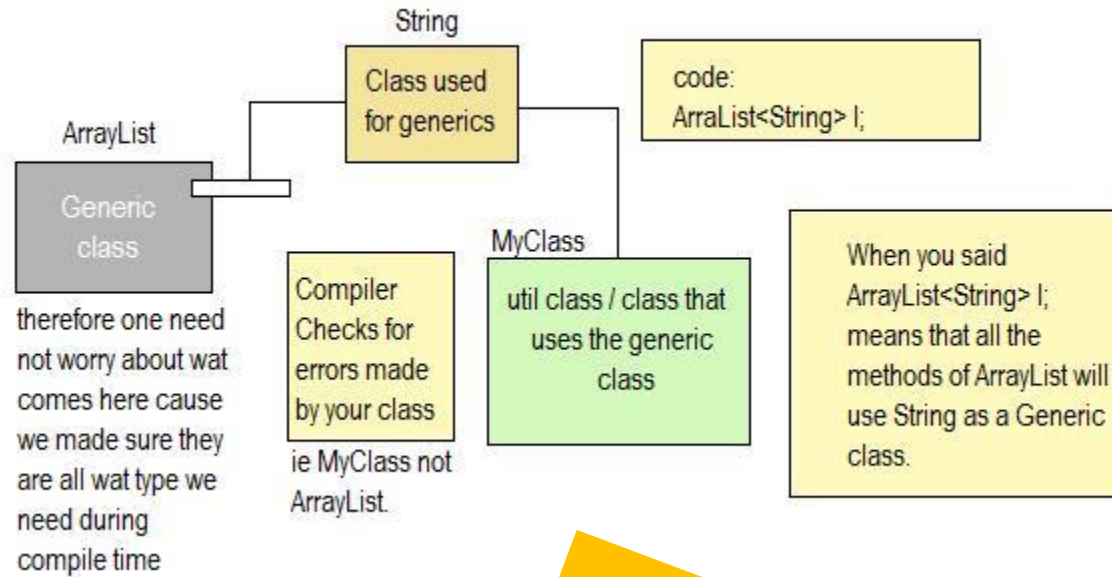


Generics



Generic Methods

Generic Classes

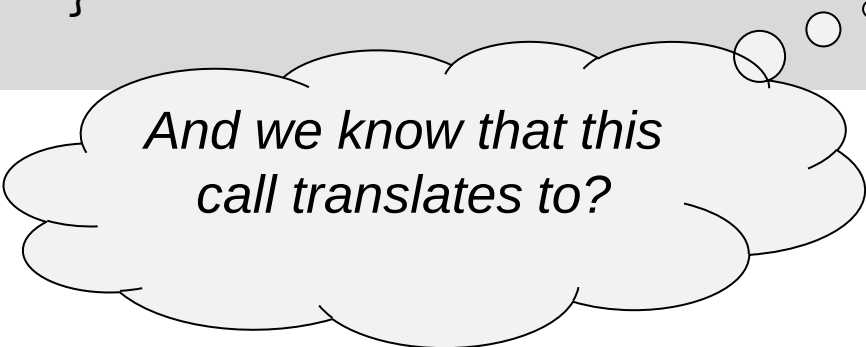
Generic Interfaces

Generics

- Generics is the capability to parameterize datatypes.
- Allows us to define a *method* (or a *class* or an *interface*) with a *generic type* that the compiler will substitute with a concrete type.

Generic Methods

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
  
        print( strings );  
  
    }  
  
    public static void print( String[] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i] + " " );  
    }  
}
```



*And we know that this
call translates to?*

Generic Methods

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
  
        print( strings );  
  
    }  
  
    public static void print( String[] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println(list[i].toString() + " ");  
    }  
}
```

Generic Methods

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
        Integer[] integers = {1, 2, 3, 4, 5};  
  
        print( strings );  
        print( integers ); // No, print is expecting a String  
    }  
  
    public static void print( String[] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i] + " " );  
    }  
}
```

Generic Methods:

overload the method?

```
public class GenericMethod {
```

```
    public static void main(  
        String [] strings = {
```

```
        Integer[] integers = {
```

```
        print( strings );
```

```
        print( integers );
```

```
    }
```

```
    public static void print( String[] list ) {
```

```
        for ( int i = 0; i < list.length; i++ )
```

```
            System.out.println( list[i] + " " );
```

```
    }
```

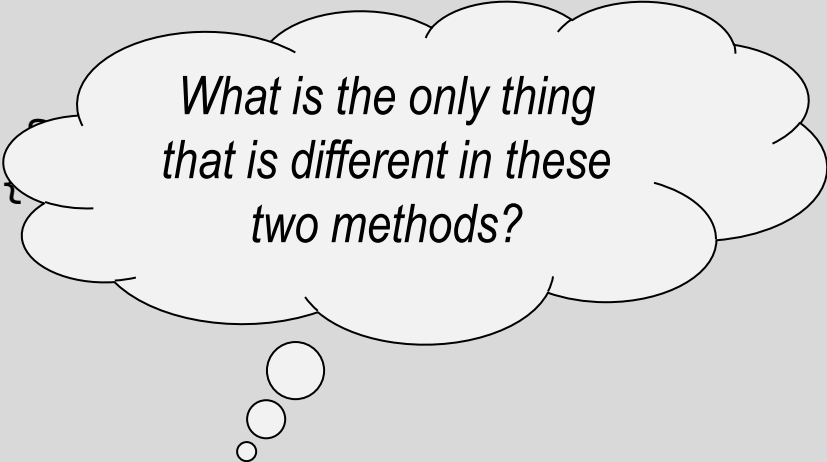
```
    public static void print( Integer[] list ) {
```

```
        for ( int i = 0; i < list.length; i++ )
```

```
            System.out.println( list[i] + " " );
```

```
    }
```

```
}
```



*What is the only thing
that is different in these
two methods?*

Generic Methods:

overload the method?

```
public class GenericMethod {
```

```
    public static void main(
```

```
        String [] strings = {
```

```
            Integer[] integers =
```

```
                print( strings );
```

```
                print( integers );
```

```
    }
```

```
    public static void print( String[] list ) {
```

```
        for ( int i = 0; i < list.length; i++ )
```

```
            System.out.println( list[i] + " " );
```

```
    }
```

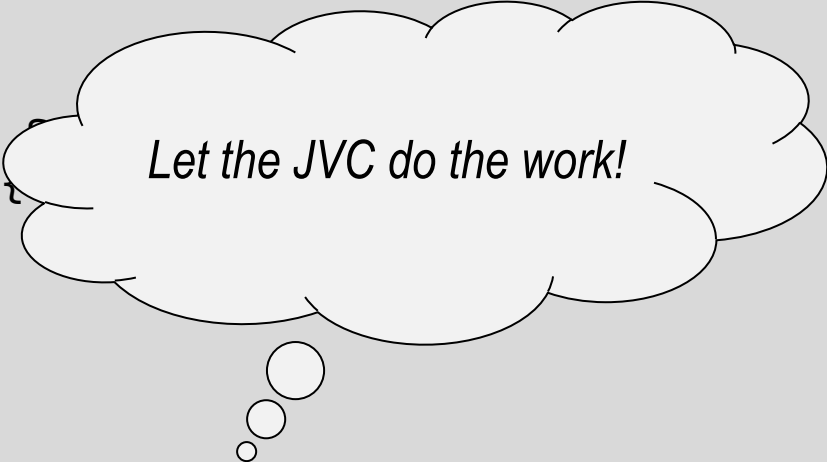
```
    public static void print( Integer[] list ) {
```

```
        for ( int i = 0; i < list.length; i++ )
```

```
            System.out.println( list[i] + " " );
```

```
    }
```

```
}
```



Let the JVC do the work!

Generic Methods:

define a generic method

```
public class GenericMethod {
```

```
    public static void main(
```

```
        String [] strings = {
```

```
            Integer[] integers = {
```

```
                print( strings );
```

```
                print( integers );
```

```
    }
```

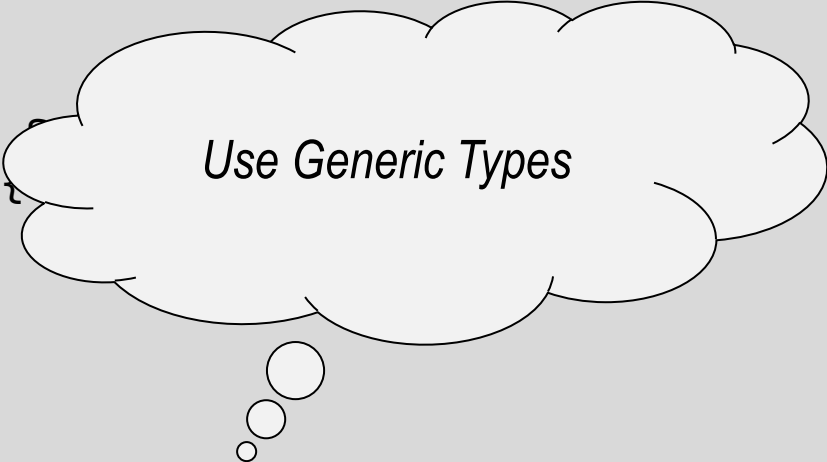
```
    public static <T> void print( T[] list ) {
```

```
        for ( int i = 0; i < list.length; i++ )
```

```
            System.out.println( list[i] + " " );
```

```
    }
```

```
}
```



Use Generic Types

Generic Methods:

define a generic method

```
public class GenericMethod {
```

```
    public static void main(
```

```
        String [] strings = {
```

```
            Integer[] integers = {
```

```
                print( strings );
```

```
                print( integers );
```

```
    }
```

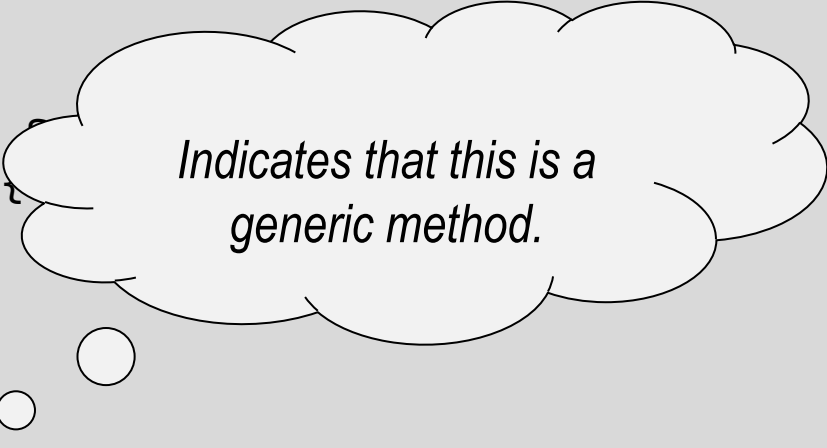
```
    public static <T> void print( T[] list ) {
```

```
        for ( int i = 0; i < list.length; i++ )
```

```
            System.out.println( list[i] + " " );
```

```
    }
```

```
}
```



*Indicates that this is a
generic method.*

Generic Methods:

define a generic method

```
public class GenericMethod {
```

```
    public static void main(
```

```
        String [] strings = {
```

```
            Integer[] integers = {
```

```
                print( strings );
```

```
                print( integers );
```

```
    }
```

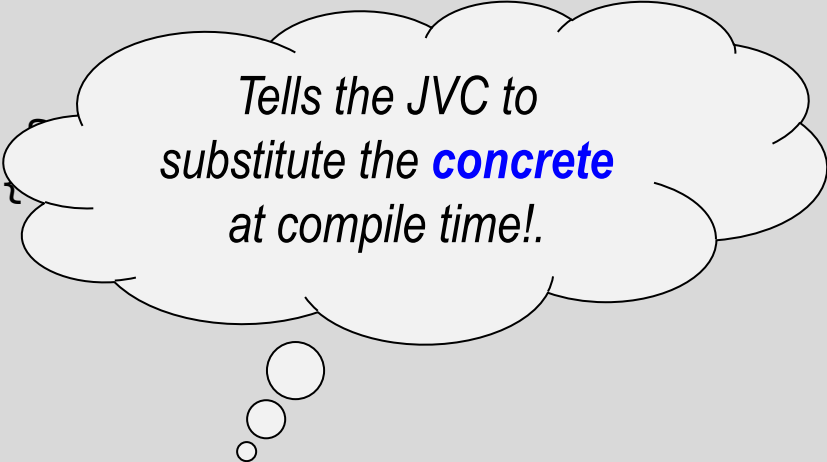
```
    public static <T> void print( T[] list ) {
```

```
        for ( int i = 0; i < list.length; i++ )
```

```
            System.out.println( list[i] + " " );
```

```
    }
```

```
}
```



*Tells the JVC to
substitute the **concrete**
at compile time!.*

Generic Methods:

define a generic method

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
        Integer[] integers = {1, 2, 3, 4, 5};  
  
        print( strings );  
        print( integers );  
    }  
  
    public static <T> void print( T[] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i] + " " );  
    }  
}
```

Generic Methods:

define a generic method

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
        Integer[] integers = {1, 2, 3, 4, 5};  
  
        print( strings );  
        print( integers );  
    }  
  
    public static <T> void print( String [] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i] + " " );  
    }  
}
```

Generic Methods:

define a generic method

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
        Integer[] integers = {1, 2, 3, 4, 5};  
  
        print( strings );  
        print( integers );  
    }  
  
    public static <T> void print( T[] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i] + " " );  
    }  
}
```

Generic Methods:

define a generic method

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
        Integer[] integers = {1, 2, 3, 4, 5};  
  
        print( strings );  
        print( integers );  
    }  
  
    public static <T> void print( Integer [] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i] + " " );  
    }  
}
```

Generic Methods:

a final note

```
public class GenericMethod {  
  
    public static void main( String [] argv ) {  
        String [] strings = { "Boston", "Chicago", "NYC" };  
        Integer[] integers = {1, 2, 3, 4, 5};  
  
        print( strings );  
        print( integers );  
    }  
  
    public static <T> void print( Integer [] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i] + " " );  
    }  
}
```

Generic Methods:

a final note

*Can this generic method be called
on arrays of any type?*

```
public class GenericMethod {  
    public static void  
        String [] strings  
        Integer [] integers  
  
        GenericMethod.<String>print( strings );  
        GenericMethod.<Integer>print( integers );  
}  
  
public static <T> void print( T [] list ) {  
    for ( int i = 0; i < list.length; i++ )  
        System.out.println( list[i] + " " );  
}  
}
```


Generic Methods:

a fine

*Assumes that the
toString method
has been implemented for
this type.*

```
public class GenericMethod {  
    public static void main(  
        String [] strings  
        Integer[] integers  
  
        GenericMethod.<String>print( strings ),  
        GenericMethod.<Integer>print( integers );  
    }  
  
    public static <T> void print( T [] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i].toString() + " " );  
    }  
}
```

Generic Methods:

a fine

Can we guarantee that
all objects have an
implementation of this
method?

```
public class GenericMethod {  
    public static void main(  
        String [] strings  
        Integer[] integers  
  
        GenericMethod.<String>print( strings ),  
        GenericMethod.<Integer>print( integers );  
    }  
  
    public static <T> void print( T [] list ) {  
        for ( int i = 0; i < list.length; i++ )  
            System.out.println( list[i].toString() + " " );  
        list[i].someMethod();  
    }  
}
```

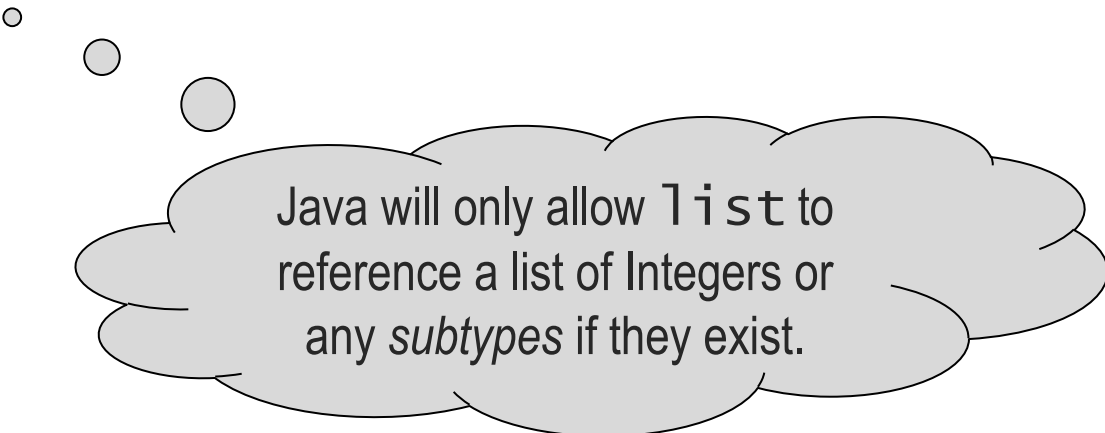
Generic Types in Java

At the core of generics is the concept of “**type safety**”.

A guarantee by the compiler that if correct Types are used in correct places, then there should **not be** any `ClassCastException` in runtime.

Example:

```
List<Integer> list;
```



Java will only allow `list` to reference a list of `Integers` or any *subtypes* if they exist.

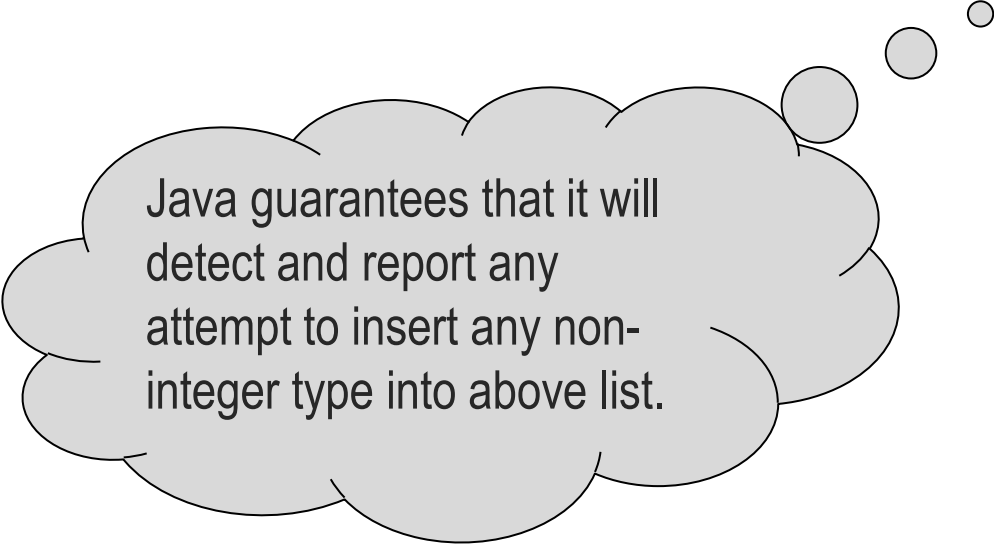
Generic Types in Java

At the core of generics is the concept of “**type safety**”.

A guarantee by the compiler that if correct Types are used in correct places, then there should **not be** any `ClassCastException` in runtime.

Example:

```
List<Integer> list = new List<Integer>();
```



Java guarantees that it will detect and report any attempt to insert any non-integer type into above list.

Bounded Type

```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Rectangle r = new Rectangle(2, 2);  
        Circle c = new Circle(2);  
  
        if ( equalArea( r, c ) )  
            System.out.println( r + c + "equal area" );  
    }  
  
    public static <T> boolean equalArea( T obj1, T obj2 ) {  
        return obj1.area() == obj2.area();  
    }  
}
```

Bounded Type

The only concrete type that should be allowed to be substituted is a type for objects that have implemented the `area()` method.

```
public class Gener
    public sta
        Rectangl
        Circle

    if ( equalArea( r, c ) )
        System.out.print( r + c + "equal area" );
    }

    public static <T> boolean equalArea( T obj1, T obj2 ) {
        return obj1.area() == obj2.area();
    }
}
```

Bounded Type

```
public class Generi
```

```
public sta
```

```
    Rectangl
```

```
    Circle
```

```
    if ( equalArea( r, c ) )
```

```
        System.out.print( r + c + "equal area" );
```

```
    }
```

```
public static <T extends Shape> boolean
```

```
    equalArea( T obj1, T obj2 ) {
```

```
        return obj1.area() == obj2.area();
```

```
    }
```

```
}
```

*The bounded generic type specifies that **T** is a generic subtype of Shape, and you must invoke this method with two instances of Shapes.*

Bounded Type

```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Rectangle r = new Rectangle(2, 2);  
        Circle c = new Circle(2);  
  
        if ( equalArea( r, c ) )  
            System.out.println( r + c + "equal area" );  
    }  
  
    public static boolean  
        equalArea( Shape obj1, Shape obj2 ) {  
        return obj1.area() == obj2.area();  
    }  
}
```


Bounded Type

```
public class Generi
```

```
public sta
```

```
Rectangl
```

```
Circle
```

```
if ( equalArea( r, c ) )
```

```
System.out.print( r + c + "equal area" );
```

```
}
```

```
public static Shape
```

```
equalArea( Shape obj1, Shape obj2 ) {
```

```
return obj1.area() == obj2.area();
```

```
}
```

```
}
```

*What if you wanted consistency
between the type of the object
passed to the method and the type of
the object returned?*

Bounded Type

```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Rectangle r1 = new Rectangle(2, 2);  
        Rectangle r2 = new Rectangle(5, 8);  
  
        Rectangle r3 = someMethod( r1, r2 );  
  
    }  
  
    public static Shape  
        someMethod( Shape obj1, Shape obj2 ) {  
        ...  
        ...  
  
        return (obj1);  
    }  
}
```

Bounded Type

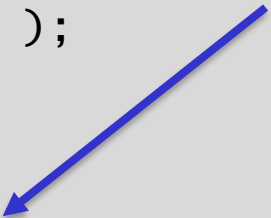
```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Rectangle r1 = new Rectangle(2, 2);  
        Rectangle r2 = new Rectangle(5, 8);  
  
        Rectangle r3 = (Rectangle) someMethod( r1, r2 );  
  
    }  
  
    public static Shape  
        someMethod( Shape obj1, Shape obj2 ) {  
        ...  
        ...  
  
        return (obj1);  
    }  
}
```

Bounded Type

```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Circle c1 = new Circle(2);  
        Circle c2 = new Circle(4);;  
  
        Circle c3 = (Circle) someMethod( c1, c2 );  
  
    }  
  
    public static Shape  
        someMethod( Shape obj1, Shape obj2 ) {  
        ...  
        ...  
  
        return (obj1);  
    }  
}
```

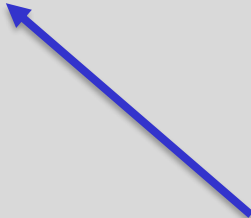
Bounded Type

```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Circle c1 = new Circle(2);  
        Circle c2 = new Circle(4);;  
  
        Circle c3 = someMethod( c1, c2 );  
  
    }  
  
    public static <T extends Shape> T  
        someMethod( T obj1, T obj2 ) {  
        ...  
        ...  
  
        return (obj1);  
    }  
}
```



Bounded Type

```
public class GenericMethod_2 {  
    public static void main( String [] argv ) {  
        Rectangle r = new Rectangle(5,4);  
        Circle c = new Circle(4);;  
  
        ? x = someMethod( r, c );  
    }  
  
    public static <T extends Shape> T  
        someMethod( T obj1, T obj2 ) {  
        ...  
        ...  
  
        return (obj1);  
    }  
}
```



Bounded Type

```
public class GenericMethod_2 {  
    public static void main( String [] argv ) {  
        Rectangle r = new Rectangle(5,4);  
        Circle c = new Circle(4);;
```

```
        Circle x = someMethod( r, c );
```

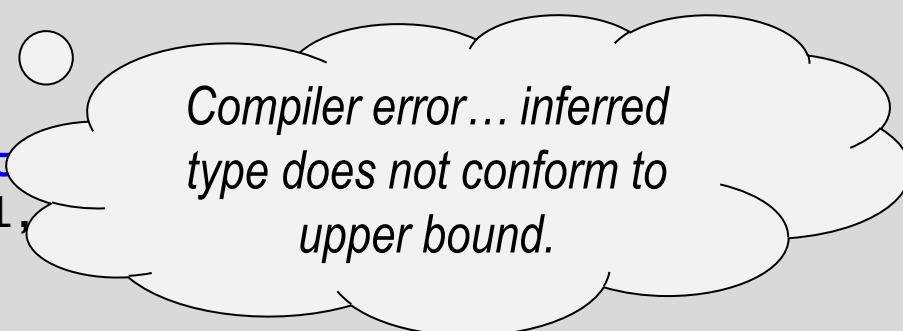
```
    }
```

```
    public static <T extends  
        someMethod( T obj1,  
        ...  
        ...
```

```
        return (obj1);
```

```
    }
```

```
}
```



*Compiler error... inferred
type does not conform to
upper bound.*

Bounded Type

```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Rectangle r = new Rectangle(5,4);  
        Circle c = new Circle(4);;  
  
        Rectangle x = someMethod( r, c );  
    }  
  
    public static <T extends Rectangle> someMethod( T obj1,  
        ...  
        ...  
  
        return (obj1);  
    }  
}
```

*Compiler error... inferred
type does not conform to
upper bound.*

Bounded Type

```
public class GenericMethod_2 {  
    public static void main( String [] argv ) {  
        Rectangle r = new Rectangle(5,4);  
        Circle c = new Circle(4);;
```

```
        Shape x = someMethod( r, c );
```

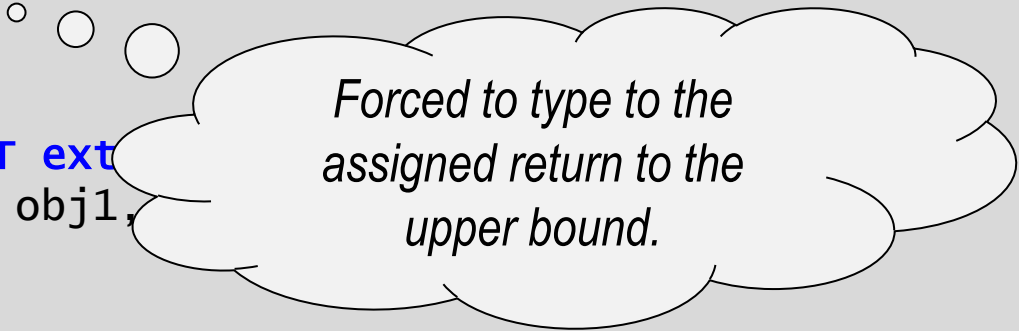
```
    }
```

```
    public static <T extends  
        someMethod( T obj1,  
        ...  
        ...
```

```
        return (obj1);
```

```
    }
```

```
}
```



*Forced to type to the
assigned return to the
upper bound.*

Bounded Type

```
public class GenericMethod_2 {  
    public static void main( String [] argv ) {  
        Circle c1 = new Circle(5);  
        Circle c2 = new Circle(4);;  
  
        Circle x = someMethod( c1, c2 );  
    }  
  
    public static <T extends Shape> T  
        someMethod( T obj1, T obj2 ) {  
        ...  
        ...  
  
        return (obj1);  
    }  
}
```

Un-Bounded Type

```
public class GenericMethod_2 {  
  
    public static void main( String [] argv ) {  
        Rectangle r = new Rectangle(2, 2);  
        Circle c = new Circle(2);  
  
        if ( equalArea( r, c ) )  
            System.out.println( r + c + "equal area" );  
    }  
  
    public static <T> boolean  
        equalArea( T obj1, T obj2 ) {  
        return obj1.area() == obj2.area();  
    }  
}
```

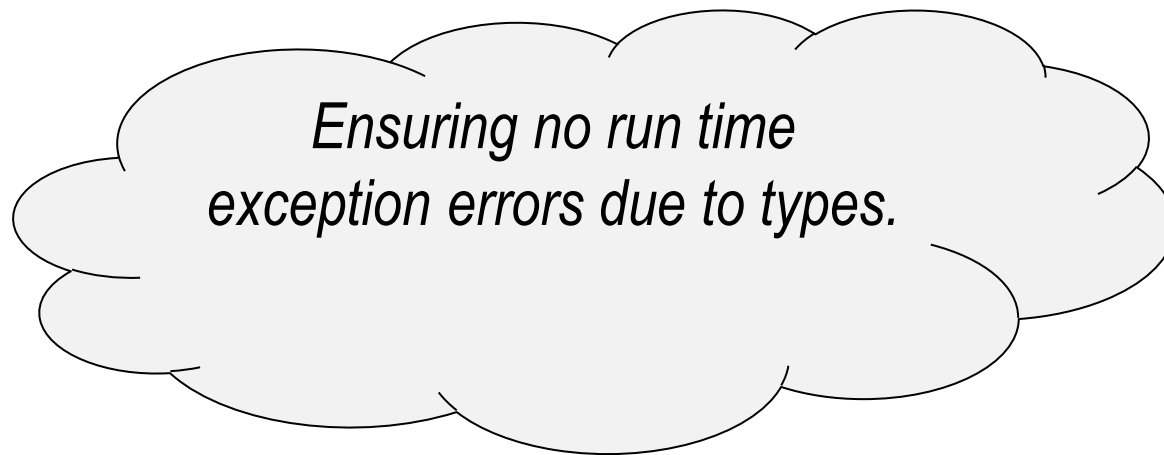
Bounded Type

```
public class GenericMethod_2 {  
    public static  
        Rectan  
        Circle  
        if ( equalArea( r, c ) )  
            System.out.println( r + c + "equal area" );  
}  
  
public static <T extends Object> boolean  
    equalArea( T obj1, T obj2 ) {  
        return obj1.area() == obj2.area();  
    }  
}
```

*The unbounded generic type
by default is bounded to the
object class!*

Generics

- By using *bounded types* we can specify allowable types and objects that the class or method can work with. If you attempt to use an incompatible type, the *compiler* can detect it.



Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable {  
    public int compareTo( Object o );  
}
```

```
public class Date implements Comparable {  
    ...  
  
    public int compareTo( Object other ) {  
        ...  
    }  
  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable {  
    public int compareTo( Object o );  
}
```

```
public class testClass {  
    public static void main( String [] a ) {  
        Date d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```



*This call should not be allowed!
Cannot compare a *Date* object to
a *String* object.*

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable {  
    public int compareTo( Object o );  
}
```

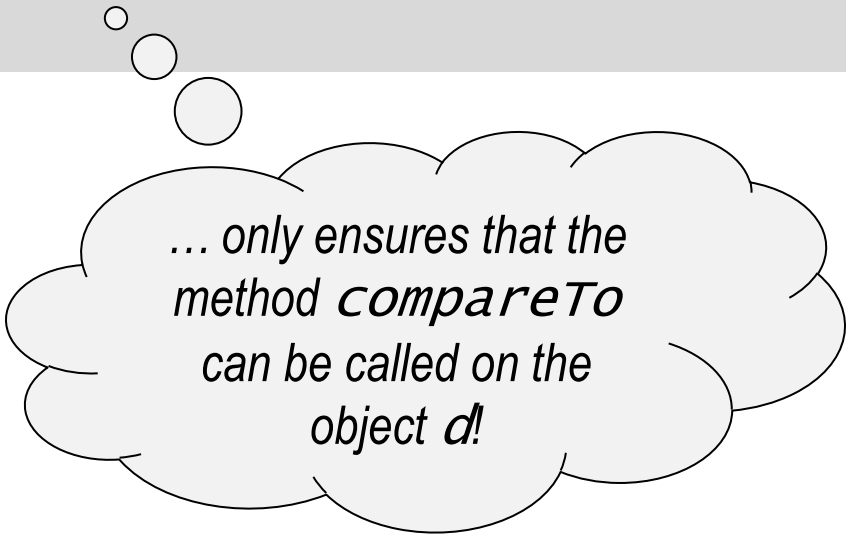
```
public class testClass {  
    public static void main( String [] a ) {  
        Comparable d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```


Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable {  
    public int compareTo( Object o );  
}
```

```
public class testClass {  
    public static void main( String [] a ) {  
        Comparable d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```

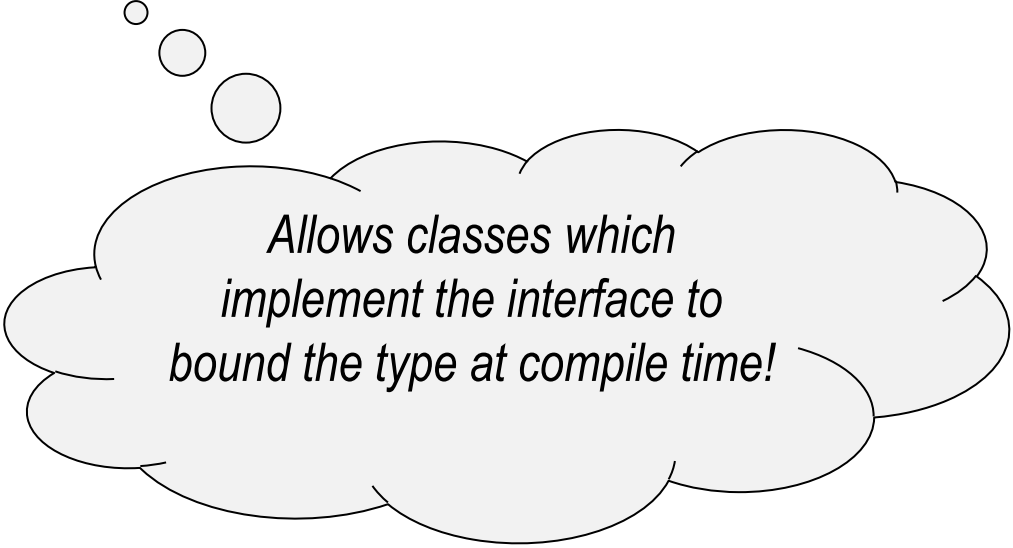


*... only ensures that the
method `compareTo`
can be called on the
object *d*!*

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```



*Allows classes which
implement the interface to
bound the type at compile time!*

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable  
{  
    ...  
  
    public int compareTo( Object other ) {  
        ...  
    }  
  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<object>  
{  
    ...  
    public int compareTo( object other ) {  
        ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Object other ) {  
        ...  
    }  
  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
  
}
```

```
public class testClass {  
    public static void main( String [] a ) {  
        Date d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
}
```

*Compiler can now detect
that we are passing an
incompatible type to the
compareTo method of
the Date class*

```
public class testClass {  
    public static void main( String a ) {  
        Date d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
  
}
```

```
public class testClass {  
    public static void main( String [] a ) {  
        Comparable<Date> d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```


Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
}
```

*Compiler can again detect
that we are passing an
incompatible type to the
compareTo method of
the Date class!*

```
public class testClass {  
    public static void main( String a ) {  
        Comparable<Date> d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
  
}
```

```
public class testClass {  
    public static void main( String [] a ) {  
        Comparable<Date> d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( new Date() ) )  
            ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
}
```

There is no compiler error here, but we would not be able to invoke any other method of the Date class on object d.

```
public class testClass {  
    public static void main( String a ) {  
        Comparable<Date> d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( new Date() ) )  
            ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
  
}
```

```
public class testClass {  
    public static void main( String [] a ) {  
        Comparable d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
  
}
```

```
public class testClass {  
    public static void main( String [] a ) {  
        Comparable<Object> d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```

Generic Interfaces

revisiting the Comparable interface

```
public interface Comparable<T> {  
    public int compareTo( T o );  
}
```

```
public class Date implements Comparable<Date>  
{  
    ...  
  
    public int compareTo( Date otherDate ) {  
        ...  
    }  
}
```

```
public class testClass {  
    public static void main( String a ) {  
        Comparable<Object> d = new Date( 7, 13, 2019 );  
  
        if ( d.compareTo( "red" ) )  
            ...  
    }  
}
```

*Potential run-time error.
There is a compiler
warning, but the Java
code would compile
(string is an Object).*

Generic Classes

the *ArrayList* class

- The Java *ArrayList* class allows you to create *array* lists of objects that are not restricted to the limitations of primitive (fixed) Java arrays.
- *ArrayList* is a Java Class *with methods to manipulate the list!*

```
+ArrayList()  
+add(o: Object)  
+clear(): void  
+contains(o: Object): boolean  
+get(index: int): Object  
+indexOf(o Object): int  
+isEmpty(): boolean  
+lastIndexOf(o: Object): int  
+remove(o: Object): boolean  
+remove(index: int): boolean  
+size(): int  
+set(index: int, o: Object): Object
```

Generic Classes

the *ArrayList* class

- The Java *ArrayList* class allows you to create *array* lists of objects that are not restricted to the limitations of primitive (fixed) Java arrays.
- *ArrayList* is a Java Class.

```
+ArrayList()  
+add(o: Object)  
+clear(): void  
+contains(o: Object): boolean  
+get(index: int): Object  
+indexOf(o Object): int  
+isEmpty(): boolean  
+lastIndexOf(o: Object): int  
+remove(o: Object): boolean  
+remove(index: int): boolean  
+size(): int  
+set(index: int, o: Object): Object
```


Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList cityList = new ArrayList();  
  
        // Create a list of cities  
        cityList.add("London");  
        cityList.add("Boston" );  
        cityList.add("NYC");  
        cityList.add("Athens");  
        cityList.add("Beijing");  
        cityList.add("Seoul");  
  
    }  
  
}
```

Generic Classes

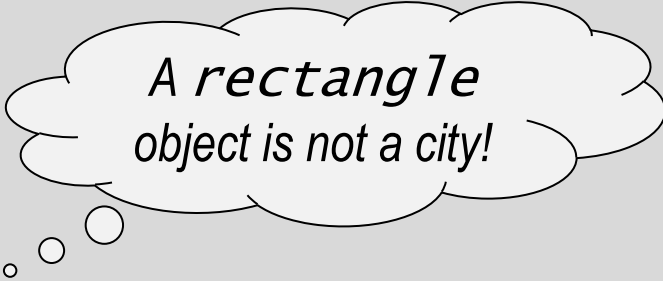
the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList cityList = new ArrayList();  
  
        // Create a list of cities  
        cityList.add("London");  
        cityList.add("Boston" );  
        cityList.add("NYC");  
        cityList.add("Athens");  
        cityList.add("Beijing");  
        cityList.add("Seoul");  
  
        cityList.remove("NYC");  
  
    }  
  
}
```

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList cityList = new ArrayList();  
  
        // Create a list of cities  
        cityList.add("London");  
        cityList.add("Boston" );  
        cityList.add("NYC");  
        cityList.add("Athens");  
        cityList.add("Beijing");  
        cityList.add("Seoul");  
  
        cityList.remove("NYC");  
  
        cityList.add( new Rectangle() );  
  
    }  
  
}
```



*A rectangle
object is not a city!*

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList cityList = new ArrayList();  
  
        // Create a list of cities  
        cityList.add("London");  
        cityList.add("Boston");  
        cityList.add("NYC");  
        cityList.add("Athens");  
        cityList.add("Beijing");  
        cityList.add("Seoul");  
  
        cityList.remove("NYC");  
  
        cityList.add( new Rectangle() );  
  
    }  
}
```



*But my list allows it
because it is a list of
objects!*

Generic Classes

the ArrayList class

```
public class TestArrayList {  
    public static void main( String [] argv ) {  
  
        ArrayList<Object> cityList = new ArrayList<Object>();  
        // Create a list of cities  
        cityList.add("London");  
        cityList.add("Boston");  
        cityList.add("NYC");  
        cityList.add("Athens");  
        cityList.add("Beijing");  
        cityList.add("Seoul");  
  
        cityList.remove("NYC");  
  
        cityList.add( new Rectangle() );  
  
    }  
}
```



*This is the default
type for ArrayLists!*

Generic Classes

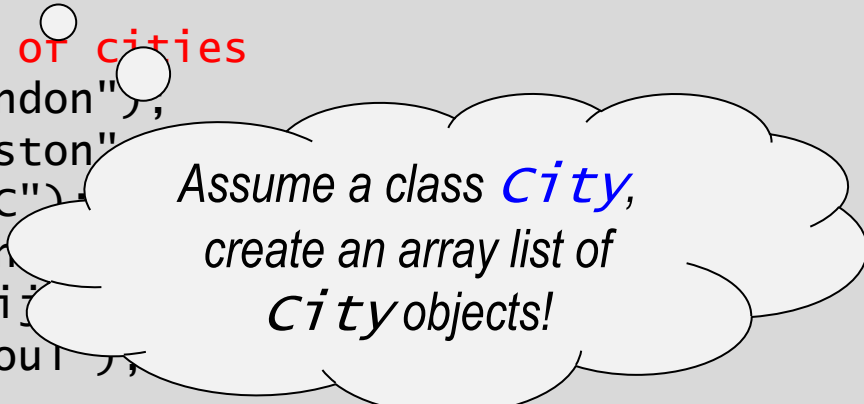
the ArrayList class

```
public class TestArrayList {  
    public static void main( String [] argv ) {  
  
        ArrayList<String> cityList = new ArrayList<String>();  
  
        // Create a list of cities  
        cityList.add("London");  
        cityList.add("Boston" );  
        cityList.add("NYC");  
        cityList.add("Athens");  
        cityList.add("Beijing");  
        cityList.add("Seoul");  
  
        cityList.remove("NYC");  
  
        cityList.add( new Rectangle() ); // compiler error!  
  
    }  
}
```

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList<City> cityList = new ArrayList<City>();  
  
        // Create a list of cities  
        cityList.add("London");  
        cityList.add("Boston");  
        cityList.add("NYC");  
        cityList.add("Ath");  
        cityList.add("Bei");  
        cityList.add("Seoul");  
  
        cityList.remove("NYC");  
  
    }  
}
```



Assume a class *City*,
create an array list of
City objects!

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList<City> cityList = new ArrayList<City>();  
  
        // Create a list of cities  
        cityList.add( new City("London") );  
        cityList.add( new City("Boston") );  
        cityList.add( new City("NYC") );  
        cityList.add( new City("Athens") );  
        cityList.add( new City("Beijing") );  
        cityList.add( new City("Seoul") );  
  
        cityList.remove("NYC"); // would this work?  
  
    }  
  
}
```


Generic Classes

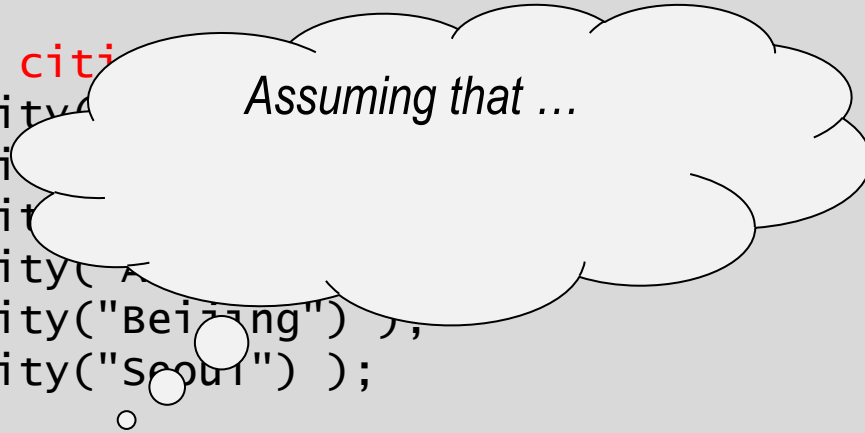
the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList<City> cityList = new ArrayList<City>();  
  
        // Create a list of cities  
        cityList.add( new City("London") );  
        cityList.add( new City("Boston") );  
        cityList.add( new City("NYC") );  
        cityList.add( new City("Athens") );  
        cityList.add( new City("Beijing") );  
        cityList.add( new City("Seoul") );  
  
        cityList.remove( new City("NYC") );  
  
    }  
  
}
```

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList<City> cityList = new ArrayList<City>();  
  
        // Create a list of cities  
        cityList.add( new City("New York") );  
        cityList.add( new City("London") );  
        cityList.add( new City("Paris") );  
        cityList.add( new City("Tokyo") );  
        cityList.add( new City("Beijing") );  
        cityList.add( new City("Seoul") );  
  
        cityList.remove( new City("NYC") );  
  
    }  
}
```

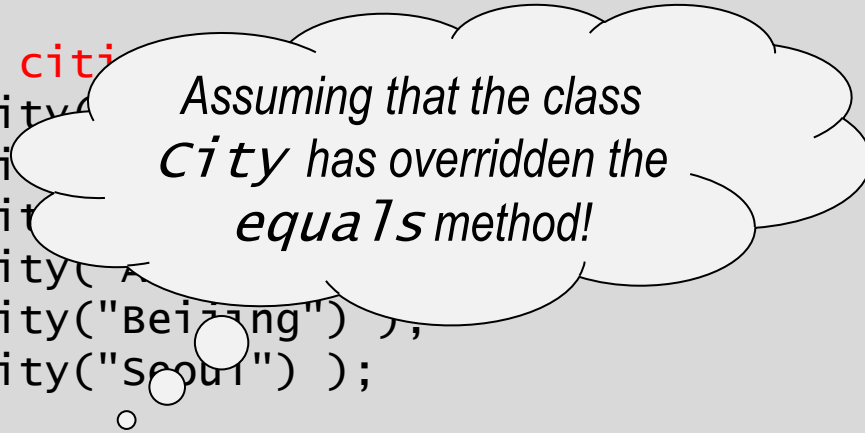


Assuming that ...

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList<City> cityList = new ArrayList<City>();  
  
        // Create a list of cities  
        cityList.add( new City("New York") );  
        cityList.add( new City("London") );  
        cityList.add( new City("Paris") );  
        cityList.add( new City("Tokyo") );  
        cityList.add( new City("Beijing") );  
        cityList.add( new City("Seoul") );  
  
        cityList.remove( new City("NYC") );  
  
    }  
}
```



Assuming that the class
City has overridden the
equals method!

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList<City> cityList = new ArrayList<City>();  
  
        // Create a list of cities  
        cityList.add( new City("London") );  
        cityList.add( new City("Boston") );  
        cityList.add( new City("NYC") );  
        cityList.add( new City("Athens") );  
        cityList.add( new City("Beijing") );  
        cityList.add( new City("Seoul") );  
  
        cityList.remove( new City("NYC") );  
  
        cityList.add("NYC"); // compile time error  
  
    }  
  
}
```

Generic Classes

the ArrayList class

```
public class TestArrayList {  
  
    public static void main( String [] argv ) {  
  
        ArrayList<City> cityList = new ArrayList<City>();  
  
        // Create a list of cities  
        cityList.add( new City("London") );  
        cityList.add( new City("Boston") );  
        cityList.add( new City("NYC") );  
        cityList.add( new City("Athens") );  
        cityList.add( new City("Beijing") );  
        cityList.add( new City("Seoul") );  
  
        cityList.remove( new City("NYC") );  
  
        cityList.add(new Rectangle()); // compile time error  
  
    }  
  
}
```

Generic Classes

the *ArrayList* class

- The Java *ArrayList* is a Generic class in Java (beginning with version 1.5 of Java).

```
+ArrayList()  
+add(o: E): void  
+clear(): void  
+contains(o: E): boolean  
+get(index: int): Object  
+indexOf(o E): int  
+isEmpty(): boolean  
+lastIndexOf(o: E): int  
+remove(o: E): boolean  
+remove(index: int): boolean  
+size(): int  
+set(index: int, o: E): E
```

A Generic Stack Class



Limiting a Stack to Objects of a Given Type

- A *generic* interface and class.

- Here's a generic version of our Stack interface:

```
public interface Stack<T> {  
    boolean push(T item);  
    T pop();  
    T peek();  
    boolean isEmpty();  
    boolean isFull();  
}
```

- It includes a *type variable* **T** in its header and body.
 - used as a placeholder for the actual type of the items

A Generic ArrayStack Class

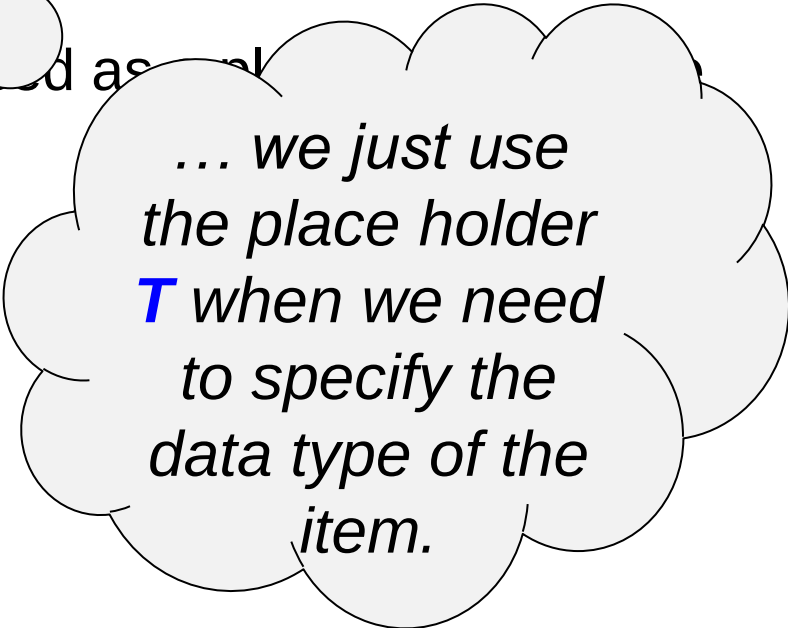
```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...  
    public boolean push(T object) {  
        ...  
    }  
    ...  
}
```

- Once again, a type variable **T** is used as a placeholder for the actual type of the items.

A Generic ArrayStack Class

```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...  
    public boolean push(T object) {  
        ...  
    }  
    ...  
}
```

- Once again, a type variable **T** is used as a placeholder for the actual type of the items.



*... we just use
the place holder
T when we need
to specify the
data type of the
item.*

If Java Generics worked like C++ templates

```
public class ArrayStack<String> {  
    private String[] items;  
    private int top;  
    ...  
    public boolean push(String item) {  
        ...  
    }  
}
```

```
ArrayStack<String> s1 =  
    new ArrayStack<String>(10);
```

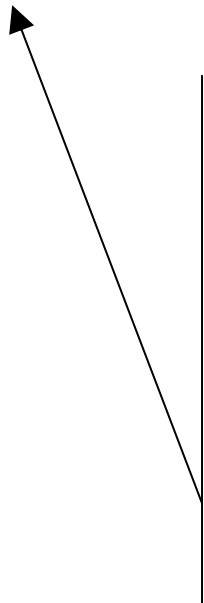
```
public class ArrayStack<T> ... {  
    private T[] items;  
    private int top;  
    ...  
    public boolean push(T item) {  
        ...  
    }  
}
```

```
ArrayStack<Integer> s1 =  
    new ArrayStack<Integer>(25);
```

```
public class ArrayStack<Integer> {  
    private Integer[] items;  
    private int top;  
    ...  
    public boolean push(Integer item) {  
        ...  
    }  
}
```

Generic Types in Java

- Java implements generics using “*type erasure*”.
- Essentially all the extra information added using generics into source code is removed from the bytecode generated from it.



The Java compiler uses the generic type information to compile the code but then erases it afterward. As a result, the information is not available at run-time.

Essentially the compiler uses the type information to confirm that a generic type is used safely, but then converts to the Raw type.

Generic Types in Java

- Java implements generics using “*type erasure*”.
- Essentially all the extra information added using generics into source code is **removed** from the bytecode generated from it.
- After translation by type erasure, all information regarding type parameters and type arguments has disappeared.
- All instantiations of the same generic type share the same runtime type, namely the **raw type**.

Generic Types in Java

- Java implements generics using “*type erasure*”.
- Essentially all the extra information added using generics into source code is **removed** from the bytecode generated from it.
- After translation by type erasure, all information regarding type parameters and type arguments has disappeared.
- All instantiations of the same generic type share the same runtime type, namely the **raw type**.

Essentially the compiler *hides* all information related to type parameters and type arguments. Example:

`List<String>`, `List<Long>` are translated as type `List` in the bytecode.

Generic Erasure

- Java implements generic types by erasing type parameters.
- Essentially, the compiler *hides* all information related to type parameters and type arguments. Example:
`List<String>`, `List<Long>` are translated as type `List` in the bytecode.
- After type erasure, all generic types share the same runtime type, namely the **raw type**.

Essentially the compiler *hides* all information related to type parameters and type arguments. Example:

`List<String>`, `List<Long>` are translated as type `List` in the bytecode.

Example

```
{  
    ArrayList<String> list = new ArrayList<String>();  
    list.add("someString");  
    String s = list.get(0);  
}
```

post-compilation...

```
{  
    ArrayList list = new ArrayList();  
    list.add("someString");  
    String s = (String) (list.get(0));  
}
```


Example:

unbounded generic type

```
public static <T> void print( T[] arr ) {  
    for ( int i = 0; i<arr.length; i++ )  
        System.out.println( arr[i] );  
}
```

post-compilation...

```
public static Object void print( Object[] arr ) {  
    for ( int i = 0; i<arr.length; i++ )  
        System.out.println( arr[i].toString() );  
}
```

Example:

bounded generic type

```
public static <T extends Shape> boolean  
    equalArea( T o1, T o2 ) {  
  
  
  
  
  
  
  
  
  
}
```

post-compilation...

```
public static Shape boolean  
    equalArea( Shape o1, Shape o2 ) {  
  
    // cast as needed during compile time  
  
}
```

Generic Types in Java

An **array** is a collection of similar type of elements.

A Generic ArrayStack Class

```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...
```

```
    public ArrayStack(int maxSize) {  
        items = new T[maxSize];    // Java does not allow this  
        top = -1;  
    }
```

```
    public boolean push(T object) {  
        ...  
    }  
    ...  
}
```

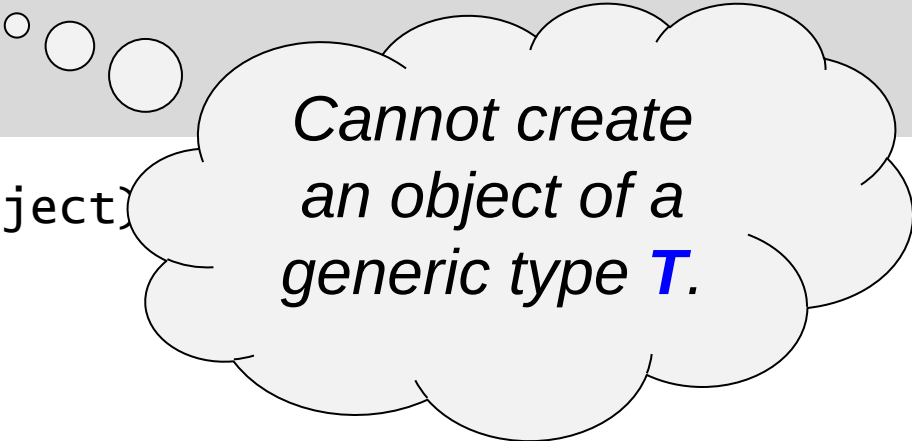
A Generic ArrayStack Class

```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...
```

```
    public ArrayStack(int maxSize) {  
        items = new T[maxSize]; // Java does not allow this  
        top = -1;  
    }
```

```
    public boolean push(T object)  
    {  
        ...  
    }  
    ...
```

```
}
```



*Cannot create
an object of a
generic type **T**.*

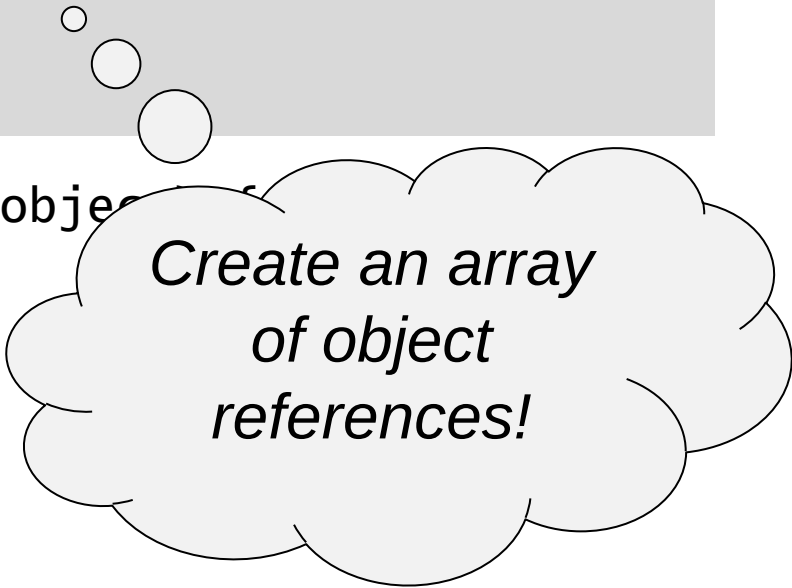
A Generic ArrayStack Class

```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...
```

```
    public ArrayStack(int maxSize) {  
        items =        new Object[maxSize];  
        top = -1;  
    }
```

```
    public boolean push(T obj) {  
        ...  
    }  
    ...
```

```
}
```



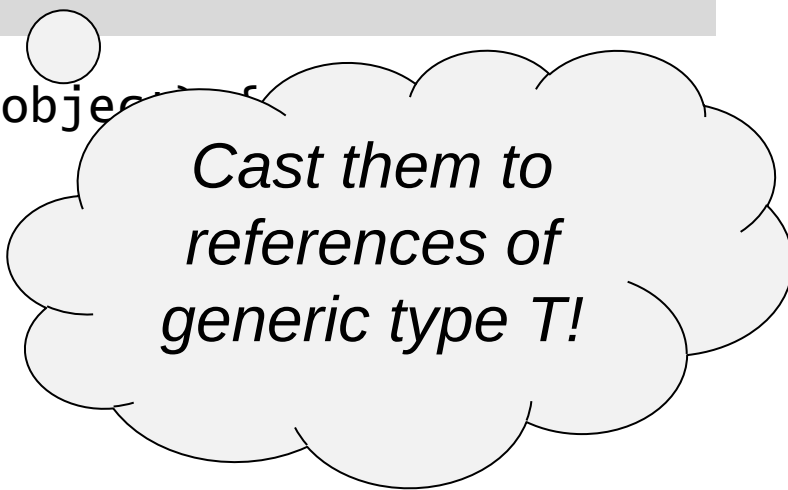
*Create an array
of object
references!*

A Generic ArrayStack Class

```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...
```

```
    public ArrayStack(int maxSize) {  
        items = (T[]) new Object[maxSize];  
        top = -1;  
    }
```

```
    public boolean push(T object) {  
        ...  
    }  
    ...  
}
```



*Cast them to
references of
generic type T!*

A Generic ArrayStack Class

```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...  
  
    public ArrayStack(int maxSize) {  
        items = (T[]) new Object[maxSize];  
        top = -1;  
    }  
  
    public boolean push(T object) {  
        ...  
    }  
    ...  
}
```

```
ArrayStack<String> s1 = new ArrayStack<String>(10);  
ArrayStack<Integers> s2 = new ArrayStack<Integers>(50);  
  
ArrayStack<Objects> s3 = new ArrayStack<Objects>(12);
```


A Generic ArrayStack Class

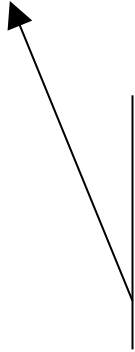
```
public class ArrayStack<T> implements Stack<T> {  
    private T[] items;  
    private int top;    // index of the top item  
    ...  
  
    public ArrayStack(int maxSize) {  
        items = (T[]) new Object[maxSize];  
        top = -1;  
    }  
  
    public boolean push(T object) {  
        ...  
    }  
    ...  
}
```

```
Stack s1 = new ArrayStack<String>(10);  
Stack s2 = new ArrayStack<Integers>(50);  
  
Stack s3 = new ArrayStack<Objects>(12);
```

Restrictions with Generic Types

1. new E()

Cannot create an *instance* of a Generic Type.



This statement is executed at run-time, but because of type erasure the generic type is not available at run-time.

Restrictions with Generic Types

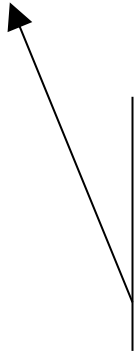
1. new E()

Cannot create an *instance* of a Generic Type.

2. new E[]

Cannot create an array using a Generic Type.

```
E[] elements = (E[]) new Object[maxNum];
```



Note: This may cause a unchecked compiler warning because the compiler cannot be certain that the casting will succeed at run-time.

Restrictions with Generic Types

1. `new E()`

Cannot create an *instance* of a Generic Type.

2. `new E[]`

Cannot create an array using a Generic Type.

```
E[] elements = (E[]) new Object[maxNum];
```

3. A generic type parameter of a class is not allowed in a static context (within the class).

Restrictions with Generic Types

1. new E()

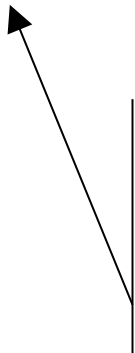
Cannot create an *instance* of a Generic Type.

2. new E[]

Cannot create an array using a Generic Type.

```
E[] elements = (E[]) new Object[maxNum];
```

3. A generic type parameter of a class is not allowed in a static context (within the class).



All instances of a generic class have the same run-time class, therefore static variables and methods of a generic class are shared by all of its instances.

Restrictions with Generic Types

1. new E()

Cannot create an *instance* of a Generic Type.

2. new E[]

Cannot create an array using a Generic Type.

```
E[] elements = (E[]) new Object[maxNum];
```

3. A generic type parameter of a class is not allowed in a static context (within the class).

```
public class testClass<E> {
```

```
}
```

Restrictions with Generic Types

1. new E()

Cannot create an *instance* of a Generic Type.

2. new E[]

Cannot create an array using a Generic Type.

```
E[] elements = (E[]) new Object[maxNum];
```

3. A generic type parameter of a class is not allowed in a static context (within the class).

```
public class testClass<E> {  
    // cannot use generic type to declare static member  
    • public static E member;  
  
}
```

Restrictions with Generic Types

1. new E()

Cannot create an *instance* of a Generic Type.

2. new E[]

Cannot create an array using a Generic Type.

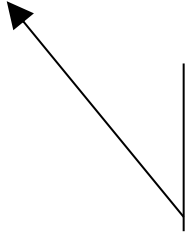
```
E[] elements = (E[]) new Object[maxNum];
```

3. A generic type parameter of a class is not allowed in a static context (within the class).

```
public class testClass<E> {  
    // cannot use generic type to declare static member  
    • public static E member;  
    // cannot use generic type in static method  
    • public static void someMethod(E param) {  
        E lvar;  
    }  
}
```


Restrictions with Generic Types

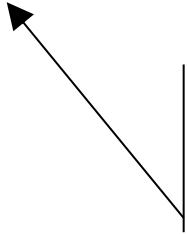
4. Exception classes cannot be generic!



You cannot pass an instance of a generic type along with the exception being thrown.

Restrictions with Generic Types

4. Exception classes cannot be generic!



You cannot pass an instance of a generic type along with the exception being thrown.

5. Cannot substitute primitive types in generic declarations. Must use the corresponding wrapper class. Example:

```
List<int> ids = new ArrayList<>(); // not allowed
```

```
List<Integer> ids = new ArrayList<>();
```

Wildcards in Generic Types

```
public class ListTest {  
  
    public static void main(String[] args) {  
        List list;  
  
  
  
  
  
  
  
  
  
    }  
}
```

Wildcards in Generic Types

```
public class ListTest {  
  
    public static void main(String[] args) {  
        List list = new ArrayList();    // ?  
  
    }  
}
```

Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List<Object> list = new ArrayList<Object>();  
    }  
}
```



Wildcards in Generic Types

```
public class ListTest {  
  
    public static void main(String[] args) {  
        List list = new ArrayList<Object>(); // ?  
  
    }  
}
```

Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List<Object> list = new ArrayList<Object>();  
    }  
}
```



Wildcards in Generic Types

```
public class ListTest {  
  
    public static void main(String[] args) {  
        List<Object> list = new ArrayList(); // ?  
  
    }  
}
```


Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List<Object> list = new ArrayList(); // warning  
    }  
}
```

Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>();  
        list = new ArrayList<Integer>();  
    }  
}
```

Wildcards in Generic Types

```
public class ListTest {  
  
    public static void main(String[] args) {  
        List list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>();  
        list = new ArrayList<Integer>();  
    }  
}
```

Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>();  
        list = new ArrayList<Integer>();  
    }  
}
```



Wildcards in Generic Types

```
public class ListTest {  
  
    public static void main(String[] args) {  
        List<Object> list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>();  
        list = new ArrayList<Integer>();  
    }  
}
```

Wildcards in Generic Types

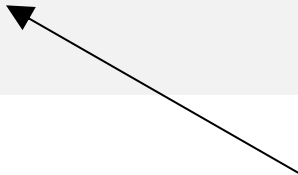
```
public class ListTest {  
    public static void main(String[] args) {  
        List<Object> list;  
  
        list = new ArrayList<Object>(); // valid assignment  
        list = new ArrayList<String>();  
        list = new ArrayList<Integer>();  
    }  
}
```

Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List<object> list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>(); // compiler error  
        list = new ArrayList<Integer>(); // compiler error  
    }  
}
```

Wildcards in Generic Types

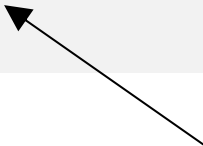
```
public class ListTest {  
    public static void main(String[] args) {  
        List<object> list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>(); // compiler error  
        list = new ArrayList<Integer>(); // compiler error  
    }  
}
```



Class String and class Integer are subtypes of Object, but ArrayList<String> and ArrayList<Integer> are not subclasses of ArrayList<Object>!

Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List<?> list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>();  
        list = new ArrayList<Integer>();  
    }  
}
```



Unbounded wildcard.... Which is

Wildcards in Generic Types

```
public class ListTest {  
    public static void main(String[] args) {  
        List<? extends Object> list;  
  
        list = new ArrayList<Object>();  
        list = new ArrayList<String>();  
        list = new ArrayList<Integer>();  
    }  
}
```

*Allows all
Object
types and
subtypes.*

Bounded to the Object type, more specifically it sets the upper bound to be class Object.

Wildcards in Generic Types

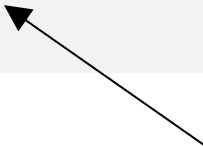
```
public class ListTest {  
  
    public static void main(String[] args) {  
        List<? extends Number> list;  
  
        list = new ArrayList<Integer>();  
        list = new ArrayList<Float>();  
        list = new ArrayList<Double>();  
    }  
}
```

*Only allow
Numeric
types?*

Bounded to the Number type, more specifically it sets the upper bound to be class Number.

Wildcards in Generic Types

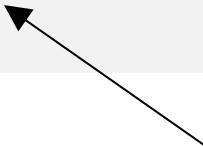
```
public class ListTest {  
    public static void main(String[] args) {  
        List<? extends Number> list;  
  
        list = new ArrayList<Integer>();  
        list = new ArrayList<Float>();  
        list = new ArrayList<String>(); // compiler error  
    }  
}
```



Bounded to the Number type, more specifically it sets the **upper bound** to be class Number.

Wildcards in Generic Types

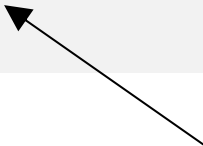
```
public class ListTest {  
    public static void main(String[] args) {  
        List<? super Integer> list;  
  
        list = new ArrayList<Integer>();  
        list = new ArrayList<Float>(); // compiler error  
        list = new ArrayList<Double>(); // compiler error  
    }  
}
```



Bounded to the Integer type, more specifically sets the **lower bound** to be class Integer.

Wildcards in Generic Types

```
public class ListTest {  
  
    public static void main(String[] args) {  
        List<? super Integer> list;  
  
        list = new ArrayList<Integer>();  
        list = new ArrayList<Number>();  
        list = new ArrayList<Object>();  
    }  
}
```



Bounded to the Integer type, more specifically sets the **lower bound** to be class Integer.

Generic Methods to *only* vary parameter types

```
public class GenericsTester {  
  
    public static double sum(? numberlist) {  
        double sum = 0.0;  
        for (Number n : numberlist) sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

Generic Methods to *only* vary parameter types

```
public class GenericsTester {  
  
    public static double sum(List<Number> numberlist) {  
        double sum = 0.0;  
        for (Number n : numberlist) sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

~

Generic Methods to *only* vary parameter types

```
public class GenericsTester {  
  
    public static double sum(List<Number> numberlist) {  
        double sum = 0.0;  
        for (Number n : numberlist) sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

~

Generic Methods to *only* vary parameter types

```
public class GenericsTester {  
  
    public static double sum(List<Number> numberlist) {  
        double sum = 0.0;  
        for (Number n : numberlist) sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Number> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Number> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

~

Generic Methods to *only* vary parameter types

```
public class GenericsTester {  
  
    public static <T extends Number>  
        double sum( List<T> numberlist ) {  
        double sum = 0.0;  
        for (Number n : numberlist)  
            sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

Generic Methods to *only* vary parameter types

```
public class GenericsTester {  
  
    public static <T extends Number>  
        double sum(List<T> numberlist ) {  
        double sum = 0.0;  
        for (Number n : numberlist)  
            sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

Wildcards

alternative in Generic Methods for varying parameter types

```
public class GenericsTester {  
  
    public static double sum(List<? extend Number> numberlist) {  
        double sum = 0.0;  
        for (Number n : numberlist) sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

Generic Methods allow for use of the *generic type within the method!*

```
public class GenericsTester {  
  
    public static <T extends Number>  
        double sum(List<T> numberlist ) {  
        T someVariable;    // can only do in Generic Methods  
        double sum = 0.0;  
        for (Number n : numberlist)  
            sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

Generic Methods allow for use of the *generic type within the method!*

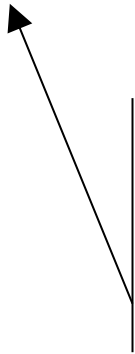
```
public class GenericsTester {  
  
    public static <T extends Number>  
        T sum(List<T> numberlist ) {  
        T someVariable;    // can only do in Generic Methods  
        double sum = 0.0;  
        for (Number n : numberlist)  
            sum += n.doubleValue();  
        return sum;  
    }  
  
    public static void main(String args[]) {  
        List<Integer> integerList = Arrays.asList(1, 2, 3);  
        System.out.println("sum = " + sum(integerList));  
  
        List<Double> doubleList = Arrays.asList(1.2, 2.3, 3.5);  
        System.out.println("sum = " + sum(doubleList));  
    }  
}
```

Wildcards and Generic types?

- The question mark **(?)**, called the wildcard, represents an unknown type.
- A wildcard parameterized type is an instantiation of a generic type where at least one type argument is a wildcard.

Unbounded Wildcard

- To declare an unbounded wildcard, simply use the wildcard character as `<?>`.
- `"?"` denotes any unknown type.



is equivalent to:

`? extends Object`

Bounded Wildcard

- To declare an upper-bounded wildcard, use the wildcard character ('?'), followed by the *extends* keyword, followed by its upper bound as in **<? extends T>**.
- All Types which are either "T" or extends T means a subtype of T.

Bounded Wildcard

- To declare an upper-bounded wildcard, use the wildcard character ('?'), followed by the *extends* keyword, followed by its upper bound as in **<? extends T>**.
- All Types which are either "T" or extends T means a subtype of T.

Example:

```
ArrayList<? extends Number>  
    nList = new ArrayList<Number>();
```

```
nList = new ArrayList<Integer>();  
nList = new ArrayList<Float>();  
nList = new ArrayList<Long>();
```

Bounded Wildcard

- To declare a *lower*-bounded wildcard, use the wildcard character ('?'), followed by the *super* keyword, followed by its upper bound as in `<? super T>`.
- This bounds allows all types which are "T" and super classes of T.

Bounded Wildcard

- To declare a lower-bounded wildcard, use the wildcard character ('?'), followed by the *super* keyword, followed by its upper bound as in `<? super T>`.
- This bounds allows all types which are "T" and super classes of T.
- **Example:**

```
ArrayList<? super Integer>  
    nList = new ArrayList<Number>();
```

```
nList = new ArrayList<Integer>();  
nList = new ArrayList<Number>();  
nList = new ArrayList<Float>();
```

Bounded Wildcard

- To declare a lower-bounded wildcard, use the wildcard character ('?'), followed by the *super* keyword, followed by its upper bound as in `<? super T>`.
- This bounds allows all types which are "T" and super classes of T.
- **Example:**

```
ArrayList<? super Integer>  
    nList = new ArrayList<Number>();  
  
nList = new ArrayList<Integer>();  
nList = new ArrayList<Number>();  
nList = new ArrayList<Float>(); // Error
```

Wildcard bound vs. Generic Type bound

Wildcard

```
? extends SuperType  
? Super SubType
```

1. A wildcard can have a lower or an upper bound.
2. A wildcard can have only one bound, while a type parameter can have several bounds.

Generic Type Bound

```
T extends Class
```

Wildcard bound vs. Generic Type bound

Wildcard

```
? extends SuperType  
? Super SubType
```

```
// upper bound  
// lower bound
```

- 1. A wildcard can have a lower or an upper bound**
2. A wildcard can have only one bound, while a type parameter can have several bounds.

Generic Type Bound

```
T extends Class
```


Wildcard bound vs. Generic Type bound

Wildcard

```
? extends SuperType  
? Super SubType
```

```
// upper bound  
// lower bound
```

1. A wildcard can have a lower or an upper bound.
2. A wildcard can have only one bound, while a type parameter can have several bounds.

Generic Type Bound

```
T extends Class & Interface1 & ... & InterfaceN
```